DEPARTAMENTO:	CIENCIAS DE LA COMPUTACIÓN	CARRERA:	INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN		
ASIGNATURA:	Modelos discretos para la ingeniería	NIVEL:	3	FECHA:	16/07/2026
DOCENTE:	Javier José Cevallos Farías	PRÁCTICA N°:	1	CALIFICACIÓN:	

Aplicación del algoritmo de Dijkstra para la determinación de rutas mínimas en grafos ponderados no dirigidos

Jeffry David Vera Borja

RESUMEN

En la presente práctica se aplica el algoritmo de Dijkstra para la búsqueda de caminos mínimos en grafos ponderados no dirigidos. Se inicia con el análisis de un grafo de ejemplo, en el que se determina la ruta más corta entre los vértices S y T, siguiendo de manera ordenada la inicialización de distancias, la selección del vértice no visitado con menor distancia tentativa, la actualización de los vértices vecinos y el registro del predecesor de cada nodo. Posteriormente se resuelve un segundo grafo compuesto por ocho vértices (A, B, C, D, E, F, G, H), determinando las rutas de menor costo desde A hacia H y desde A hacia G. El desarrollo incluye la tabla de iteraciones del algoritmo, la evolución de las distancias tentativas paso a paso, la verificación de los resultados obtenidos y una implementación en lenguaje Java que automatiza el procedimiento. Se concluye que el algoritmo de Dijkstra permite obtener de forma sistemática y verificable la ruta óptima en grafos con pesos no negativos, siendo ambas rutas encontradas (A-C-D-F-H y A-B-G) de costo mínimo igual a 8.

Palabras clave: grafos, algoritmo de Dijkstra, rutas mínimas.

1. INTRODUCCIÓN

El algoritmo de Dijkstra es uno de los métodos más relevantes de la teoría de grafos para determinar el camino de menor costo desde un vértice origen hacia los demás vértices de un grafo ponderado con pesos no negativos. Su aplicación es amplia en contextos como redes de comunicación, sistemas de posicionamiento (GPS), diseño de redes de transporte y optimización de procesos. Esta práctica busca fortalecer el razonamiento lógico y algorítmico del estudiante mediante la resolución manual y computacional de dos grafos ponderados, aplicando el procedimiento iterativo de selección de vértices, actualización de distancias tentativas y reconstrucción del camino óptimo, en el marco de la disciplina y el manejo correcto del laboratorio de computación.

2. OBJETIVO(S)

• Aplicar correctamente el algoritmo de Dijkstra para determinar caminos mínimos en grafos ponderados no dirigidos.

• Resolver de forma manual y computacional (Java) la ruta de menor costo entre distintos pares de vértices.

• Verificar la suma total de pesos del recorrido obtenido, contrastando la solución manual con la solución del programa.

3. MARCO TEÓRICO

Un grafo ponderado G = (V, E) está compuesto por un conjunto de vértices V y un conjunto de aristas E, donde cada arista tiene asociado un peso o costo. El algoritmo de Dijkstra, propuesto por Edsger W. Dijkstra en 1959, calcula la distancia mínima desde un vértice origen hacia todos los demás vértices alcanzables, siempre que los pesos de las aristas sean no negativos. Su funcionamiento se basa en un enfoque voraz (greedy): en cada iteración se selecciona el vértice no visitado con la menor distancia tentativa, se marca como visitado y se actualizan las distancias de sus vecinos si se encuentra un camino más corto a través de él. El algoritmo finaliza cuando se alcanza el vértice destino o cuando todos los vértices han sido visitados. Su complejidad computacional depende de la estructura de datos empleada para la cola de prioridad, siendo $O(V^2)$ en una implementación con matriz de adyacencia y $O((V+E) \log V)$ al emplear un montículo binario (binary heap).

4. DESCRIPCIÓN DEL PROCEDIMIENTO

Para el desarrollo de la práctica se siguió el siguiente procedimiento: (1) observación del grafo dado, identificando vértices y pesos; (2) definición del vértice origen y del vértice destino; (3) inicialización de las distancias, asignando 0 al origen e infinito (∞) a los demás vértices; (4) selección, en cada paso, del vértice no visitado con menor distancia tentativa; (5) actualización de las distancias de los vértices vecinos cuando se encontró un camino más corto, registrando siempre el vértice predecesor; (6) repetición del proceso hasta alcanzar el vértice destino o completar todos los vértices; (7) reconstrucción del camino mínimo desde el destino hacia el origen; y (8) verificación de la suma de pesos del recorrido final. Como material de apoyo se utilizó un procesador de texto, la guía de práctica proporcionada por el docente y un equipo de cómputo para la implementación y verificación del algoritmo en lenguaje Java.

4.1 Grafo de trabajo y lista de vértices

El ejercicio propuesto consiste en construir el grafo a partir de la siguiente lista de adyacencias y encontrar la ruta de menor costo de A a H, y de A a G.

Vértice	Adyacentes (peso)
A	C(1), B(3)
B	A(3), D(1), G(5)
C	A(1), D(2), F(5)
D	B(1), C(2), F(2), E(4)
E	D(4), G(2), H(1)
F	C(5), D(2), H(3)
G	B(5), E(2)
H	F(3), E(1)

El grafo resultante, no dirigido y ponderado, se muestra a continuación:

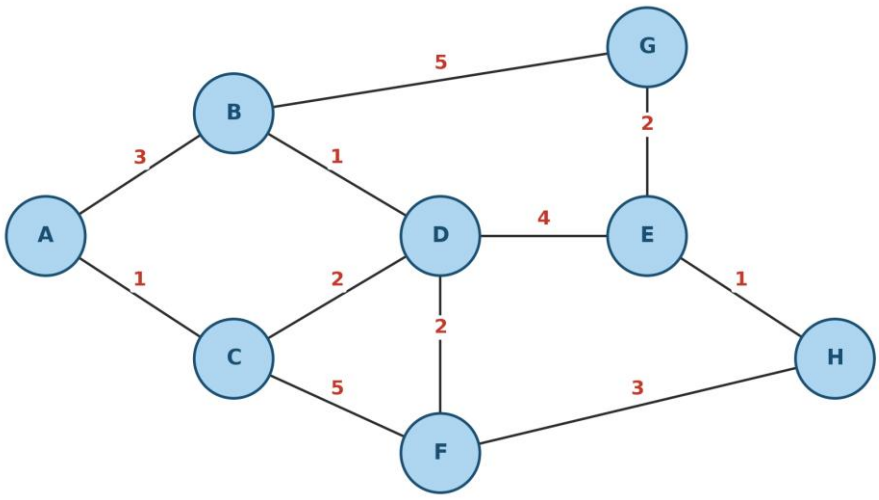


Figura 1. Grafo de trabajo con ocho vértices (A-H) y pesos asociados a cada arista.

4.2 Resolución paso a paso (Dijkstra desde A)

Se aplicó el algoritmo de Dijkstra tomando como origen el vértice A. La tabla siguiente detalla cada iteración, el vértice seleccionado, los cálculos realizados y la explicación correspondiente:

Paso	Vértice seleccionado	Distancia definitiva	Vecinos analizados	Cálculo realizado	Actualización en la tabla	Explicación
Paso 1	(A)	0	(C, B)	(A -> C = 0+1 = 1)	c = (1,A)	Se inicia en A con distancia 0. Se analiza el vecino C y se registra su distancia tentativa.
Paso 1	(A)	0	(C, B)	(A -> B = 0+3 = 3)	b = (3,A)	Se analiza el vecino B; como 3 es menor que infinito, se actualiza su distancia tentativa.
Paso 2	(C)	1	(D, F)	(1+2 = 3)	d = (3,C)	Se selecciona C por tener la menor distancia tentativa (1). Se mejora la distancia hacia D.
Paso 2	(C)	1	(D, F)	(1+5 = 6)	f = (6,C)	Desde C se llega a F con distancia acumulada 6.
Paso 3	(B)	3	(D, G)	(3+1 = 4)	d se mantiene (3,C)	B y D empatan en distancia (3); se prioriza B. El cálculo hacia D (4) no mejora el valor ya registrado (3).
Paso 3	(B)	3	(D, G)	(3+5 = 8)	g = (8,B)	Desde B se llega por primera vez a G con distancia acumulada 8.
Paso 4	(D)	3	(F, E)	(3+2 = 5)	f = (5,D)	Se selecciona D. La distancia hacia F mejora de 6 a 5.
Paso 4	(D)	3	(F, E)	(3+4 = 7)	e = (7,D)	Desde D se llega por primera vez a E con distancia acumulada 7.
Paso 5	(F)	5	(H)	(5+3 = 8)	h = (8,F)	Se selecciona F. Se llega por primera vez al vértice destino H con distancia acumulada 8.
Paso 6	(E)	7	(G, H)	(7+2 = 9) / (7+1 = 8)	g y h se mantienen	Se selecciona E. Ninguno de los cálculos (9 hacia G, 8 hacia H) mejora los valores ya registrados (8 y 8).
Paso 7	(G)	8	—	—	g = (8,B)	Se selecciona G. Al ser uno de los vértices destino, su distancia mínima queda finalizada en 8.
Paso 8	(H)	8	—	—	h = (8,F)	Se selecciona H. Al ser el último vértice destino y no quedar vértices sin visitar, el algoritmo termina.

4.3 Evolución de las distancias tentativas

La siguiente tabla resume, para cada vértice, la evolución de su distancia tentativa (valor, predecesor) a lo largo de los ocho pasos del algoritmo. El símbolo (*) indica que la distancia del vértice ya fue finalizada (visitado); las celdas resaltadas señalan el paso en que cada vértice quedó definitivamente etiquetado.

Vértice	Paso 1	Paso 2	Paso 3	Paso 4	Paso 5	Paso 6	Paso 7	Paso 8
A	0,A	*	*	*	*	*	*	*
B	3,A	3,A	3,A	*	*	*	*	*
C	1,A	1,A	*	*	*	*	*	*
D	∞	3,C	3,C	3,C	*	*	*	*
E	∞	∞	∞	7,D	7,D	7,D	*	*
F	∞	6,C	6,C	5,D	5,D	*	*	*
G	∞	∞	8,B	8,B	8,B	8,B	8,B	*
H	∞	∞	∞	∞	8,F	8,F	8,F	8,F

5.4 Verificación y resultados

Vértice	Distancia mínima desde A	Vértice anterior	Interpretación
A	0	A	Nodo inicial del recorrido.
C	1	A	Para llegar a C se viene directamente desde A.
B	3	A	Para llegar a B se viene directamente desde A.
D	3	C	Para llegar a D conviene pasar primero por C.
F	5	D	Para llegar a F conviene pasar por D.
E	7	D	Para llegar a E conviene pasar por D.
G	8	B	Vértice destino. Para llegar a G conviene pasar por B.
H	8	F	Vértice destino. Para llegar a H conviene pasar por F.

Reconstrucción de las rutas mínimas (desde el destino hacia el origen):

• Ruta A → H: H ← F ← D ← C ← A, es decir A → C → D → F → H.

Verificación de la suma de pesos: A-C(1) + C-D(2) + D-F(2) + F-H(3) = 8.

• Ruta A → G: G ← B ← A, es decir A → B → G.

Verificación de la suma de pesos: A-B(3) + B-G(5) = 8.

Ambas rutas óptimas tienen el mismo costo total (8), aunque siguen caminos completamente distintos dentro del grafo, lo cual confirma que el algoritmo evalúa cada rama de forma independiente según las distancias tentativas mínimas.

5.5 Implementación en Java

A continuación se presenta el código en Java que implementa el algoritmo de Dijkstra para el grafo A-H, utilizando una matriz de adyacencia y reconstruyendo las rutas hacia los vértices destino H y G:

```
import java.util.*;

public class DijkstraGrafoAH {

    static final int INF = Integer.MAX_VALUE;

    public static void main(String[] args) {
        // Orden de vertices: 0=A 1=B 2=C 3=D 4=E 5=F 6=G 7=H
        String[] nombres = {"A", "B", "C", "D", "E", "F", "G", "H"};
        int n = nombres.length;

        int[][] grafo = new int[n][n];
        for (int[] fila : grafo) Arrays.fill(fila, 0);

        agregarArista(grafo, 0, 2, 1);  // A-C
        agregarArista(grafo, 0, 1, 3);  // A-B
        agregarArista(grafo, 1, 3, 1);  // B-D
        agregarArista(grafo, 1, 6, 5);  // B-G
        agregarArista(grafo, 2, 3, 2);  // C-D
        agregarArista(grafo, 2, 5, 5);  // C-F
        agregarArista(grafo, 3, 5, 2);  // D-F
        agregarArista(grafo, 3, 4, 4);  // D-E
        agregarArista(grafo, 4, 6, 2);  // E-G
        agregarArista(grafo, 4, 7, 1);  // E-H
        agregarArista(grafo, 5, 7, 3);  // F-H

        int origen = 0; // A
        int[] dist = new int[n];
        int[] previo = new int[n];
        boolean[] visitado = new boolean[n];
        Arrays.fill(dist, INF);
        Arrays.fill(previo, -1);
        dist[origen] = 0;

        for (int i = 0; i < n; i++) {
            int u = verticeMenorDistancia(dist, visitado);
            if (u == -1) break;
            visitado[u] = true;
```

```
            for (int v = 0; v < n; v++) {
                if (grafo[u][v] != 0 && !visitado[v] && dist[u] != INF) {
                    int nuevaDist = dist[u] + grafo[u][v];
                    if (nuevaDist < dist[v]) {
                        dist[v] = nuevaDist;
                        previo[v] = u;
                    }
                }
            }
        }

        System.out.println("Vertice\tDistancia desde A\tAnterior");
        for (int i = 0; i < n; i++) {
            String ant = previo[i] == -1 ? "-" : nombres[previo[i]];
            System.out.println(nombres[i] + "\t" + dist[i] + "\t\t\t" + ant);
        }

        System.out.println("\nRuta A -> H: " + reconstruirRuta(previo, nombres, 7) + "  (distancia = " + dist[7] +
")");
        System.out.println("Ruta A -> G: " + reconstruirRuta(previo, nombres, 6) + "  (distancia = " + dist[6] +
")");
    }

    static void agregarArista(int[][] grafo, int u, int v, int peso) {
        grafo[u][v] = peso;
        grafo[v][u] = peso; // grafo no dirigido
    }

    static int verticeMenorDistancia(int[] dist, boolean[] visitado) {
        int min = INF, indice = -1;
        for (int i = 0; i < dist.length; i++) {
            if (!visitado[i] && dist[i] < min) {
                min = dist[i];
                indice = i;
            }
        }
        return indice;
    }

    static String reconstruirRuta(int[] previo, String[] nombres, int destino) {
        LinkedList<String> ruta = new LinkedList<>();
        for (int v = destino; v != -1; v = previo[v]) {
            ruta.addFirst(nombres[v]);
        }
        return String.join(" -> ", ruta);
    }
}

/* Salida esperada:
Vertice  Distancia desde A   Anterior
A        0                   -
B        3                   A
C        1                   A
D        3                   C
E        7                   D
F        5                   D
G        8                   B
H        8                   F

Ruta A -> H: A -> C -> D -> F -> H  (distancia = 8)
Ruta A -> G: A -> B -> G  (distancia = 8)
*/
```

5.ANÁLISIS DE RESULTADOS

5.1 Grafo de trabajo y lista de vértices

El ejercicio propuesto consiste en construir el grafo a partir de la siguiente lista de adyacencias y encontrar la ruta de menor costo de A a H, y de A a G.

Vértice	Adyacentes (peso)
A	C(1), B(3)
B	A(3), D(1), G(5)
C	A(1), D(2), F(5)
D	B(1), C(2), F(2), E(4)
E	D(4), G(2), H(1)
F	C(5), D(2), H(3)
G	B(5), E(2)
H	F(3), E(1)

El grafo resultante, no dirigido y ponderado, se muestra a continuación: *(insertar Figura 1: diagrama del grafo con los 8 vértices y sus pesos)*

5.2 Resolución paso a paso (Dijkstra desde A)

Se aplicó el algoritmo de Dijkstra tomando como origen el vértice A. A continuación se detalla cada iteración, el vértice seleccionado, los cálculos realizados y la explicación correspondiente:

- **Paso 1** – Seleccionado (A), distancia definitiva 0, vecinos analizados (C, B). Cálculo: A→C = 0+1 = 1 ⇒ c=(1,A). Se inicia en A con distancia 0 y se analiza el vecino C.
- **Paso 1** – Cálculo: A→B = 0+3 = 3 ⇒ b=(3,A). Como 3 es menor que infinito, se actualiza la distancia tentativa de B.

- **Paso 2** – Seleccionado (C), distancia definitiva 1, vecinos (D, F). Cálculo: 1+2 = 3 ⇒ d=(3,C). Se selecciona C por tener la menor distancia tentativa (1); se mejora la distancia hacia D.
- **Paso 2** – Cálculo: 1+5 = 6 ⇒ f=(6,C). Desde C se llega a F con distancia acumulada 6.
- **Paso 3** – Seleccionado (B), distancia definitiva 3, vecinos (D, G). Cálculo: 3+1 = 4 (no mejora; D se mantiene en 3,C). B y D empatan en distancia (3); se prioriza B.
- **Paso 3** – Cálculo: 3+5 = 8 ⇒ g=(8,B). Desde B se llega por primera vez a G con distancia acumulada 8.
- **Paso 4** – Seleccionado (D), distancia definitiva 3, vecinos (F, E). Cálculo: 3+2 = 5 ⇒ f=(5,D). La distancia hacia F mejora de 6 a 5.
- **Paso 4** – Cálculo: 3+4 = 7 ⇒ e=(7,D). Desde D se llega por primera vez a E con distancia acumulada 7.
- **Paso 5** – Seleccionado (F), distancia definitiva 5, vecino (H). Cálculo: 5+3 = 8 ⇒ h=(8,F). Se llega por primera vez al vértice destino H con distancia acumulada 8.
- **Paso 6** – Seleccionado (E), distancia definitiva 7, vecinos (G, H). Cálculo: 7+2=9 / 7+1=8. Ninguno de los dos cálculos mejora los valores ya registrados (8 y 8), por lo que G y H se mantienen sin cambios.
- **Paso 7** – Seleccionado (G), distancia definitiva 8. Al ser uno de los vértices destino, su distancia mínima queda finalizada en 8.
- **Paso 8** – Seleccionado (H), distancia definitiva 8. Al ser el último vértice destino y no quedar vértices sin visitar, el algoritmo termina.

5.3 Evolución de las distancias tentativas

La siguiente tabla resume, para cada vértice, la evolución de su distancia tentativa (valor, predecesor) a lo largo de los ocho pasos del algoritmo. El símbolo (*) indica que la distancia del vértice ya fue finalizada (visitado).

Vértice	Paso1	Paso2	Paso3	Paso4	Paso5	Paso6	Paso7	Paso8
A	0,A	*	*	*	*	*	*	*
B	3,A	3,A	3,A*	*	*	*	*	*
C	1,A	1,A*	*	*	*	*	*	*

D	∞	3,C	3,C	3,C*	*	*	*	*
E	∞	∞	∞	7,D	7,D	7,D*	*	*
F	∞	6,C	6,C	5,D	5,D*	*	*	*
G	∞	∞	8,B	8,B	8,B	8,B	8,B*	*
H	∞	∞	∞	∞	8,F	8,F	8,F	8,F*

5.4 Verificación y resultados

Vértice	**Distancia mínima desde A**	**Vértice anterior**	**Interpretación**
A	0	A	Nodo inicial del recorrido.
C	1	A	Para llegar a C se viene directamente desde A.
B	3	A	Para llegar a B se viene directamente desde A.
D	3	C	Para llegar a D conviene pasar primero por C.
F	5	D	Para llegar a F conviene pasar por D.
E	7	D	Para llegar a E conviene pasar por D.
G	8	B	Vértice destino. Para llegar a G conviene pasar por B.
H	8	F	Vértice destino. Para llegar a H conviene pasar por F.

Reconstrucción de las rutas mínimas (desde el destino hacia el origen):

- **Ruta A → H:** H ← F ← D ← C ← A, es decir A → C → D → F → H. Verificación de la suma de pesos: A-C(1) + C-D(2) + D-F(2) + F-H(3) = 8.
- **Ruta A → G:** G ← B ← A, es decir A → B → G. Verificación de la suma de pesos: A-B(3) + B-G(5) = 8.

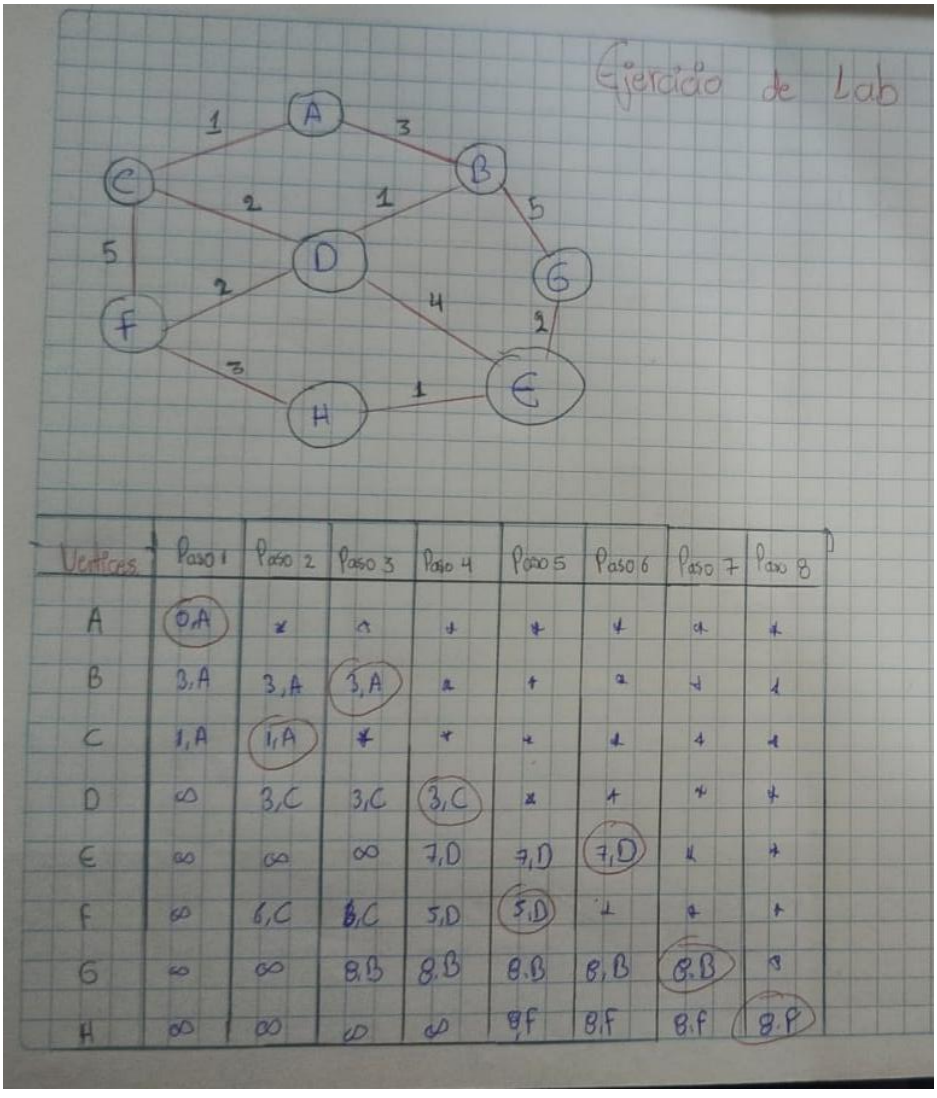
Ambas rutas óptimas tienen el mismo costo total (8), aunque siguen caminos completamente distintos dentro del grafo, lo cual confirma que el algoritmo evalúa cada rama de forma independiente según las distancias tentativas mínimas.

5.5 Implementación en Java

A continuación se presenta el código en Java que implementa el algoritmo de Dijkstra para el grafo A-H, utilizando una matriz de adyacencia y reconstruyendo las rutas hacia los vértices destino H y G *(ver código completo en el documento)*.

6. GRÁFICOS O FOTOGRAFÍAS

El diagrama del grafo (Figura 1, sección 5.1) complementa el análisis de resultados, permitiendo visualizar los vértices, las aristas y los pesos empleados durante la aplicación del algoritmo. Adicionalmente, la tabla de evolución de distancias (sección 5.3) funciona como representación gráfica del proceso iterativo de etiquetado de vértices.



Vertices	Paso 1	Paso 2	Paso 3	Paso 4	Paso 5	Paso 6	Paso 7	Paso 8
A	0,A	x	x	x	x	x	x	x
B	3,A	3,A	3,A	x	x	x	x	x
C	1,A	1,A	x	x	x	x	x	x
D	∞	3,C	3,C	3,C	x	x	x	x
E	∞	∞	∞	7,D	7,D	7,D	x	x
F	∞	6,C	6,C	5,D	5,D	x	x	x
G	∞	∞	8,B	8,B	8,B	8,B	8,B	x
H	∞	∞	∞	∞	8,F	8,F	8,F	8,F

7. DISCUSIÓN

Los resultados obtenidos de forma manual coinciden exactamente con la salida generada por el programa en Java, lo cual valida la correcta aplicación del algoritmo. Se observó que, ante un empate entre dos distancias tentativas (como ocurrió entre B y D, ambos con distancia 3), la elección del vértice a procesar no afecta el resultado final, ya que el algoritmo evalúa exhaustivamente todos los caminos posibles antes de finalizar cada vértice. Asimismo, se evidenció que un grafo puede tener más de una ruta óptima con costo mínimo idéntico hacia destinos distintos (en este caso, 8 tanto para H como para G), sin que ello implique que ambos caminos comparten aristas. Esto refuerza la importancia de registrar siempre el vértice predecesor durante la ejecución del algoritmo, pues es este registro el que permite reconstruir la ruta real y no solamente conocer el costo mínimo.

8. CONCLUSIONES

• El algoritmo de Dijkstra permitió determinar de manera sistemática la ruta de menor costo entre el vértice A y los vértices H y G, obteniendo en ambos casos un costo mínimo de 8 unidades, a través de las rutas A-C-D-F-H y A-B-G respectivamente.

• La implementación en Java replicó fielmente los resultados obtenidos en el desarrollo manual, lo que demuestra que la lógica de selección de vértice mínimo, relajación de aristas y registro de predecesores fue aplicada correctamente en ambos procedimientos.

• El uso de una tabla de evolución de distancias tentativas resultó fundamental para visualizar de manera clara el comportamiento voraz del algoritmo y para verificar, paso a paso, que cada actualización de distancia fue consistente con el peso de las aristas del grafo.

9. BIBLIOGRAFÍA

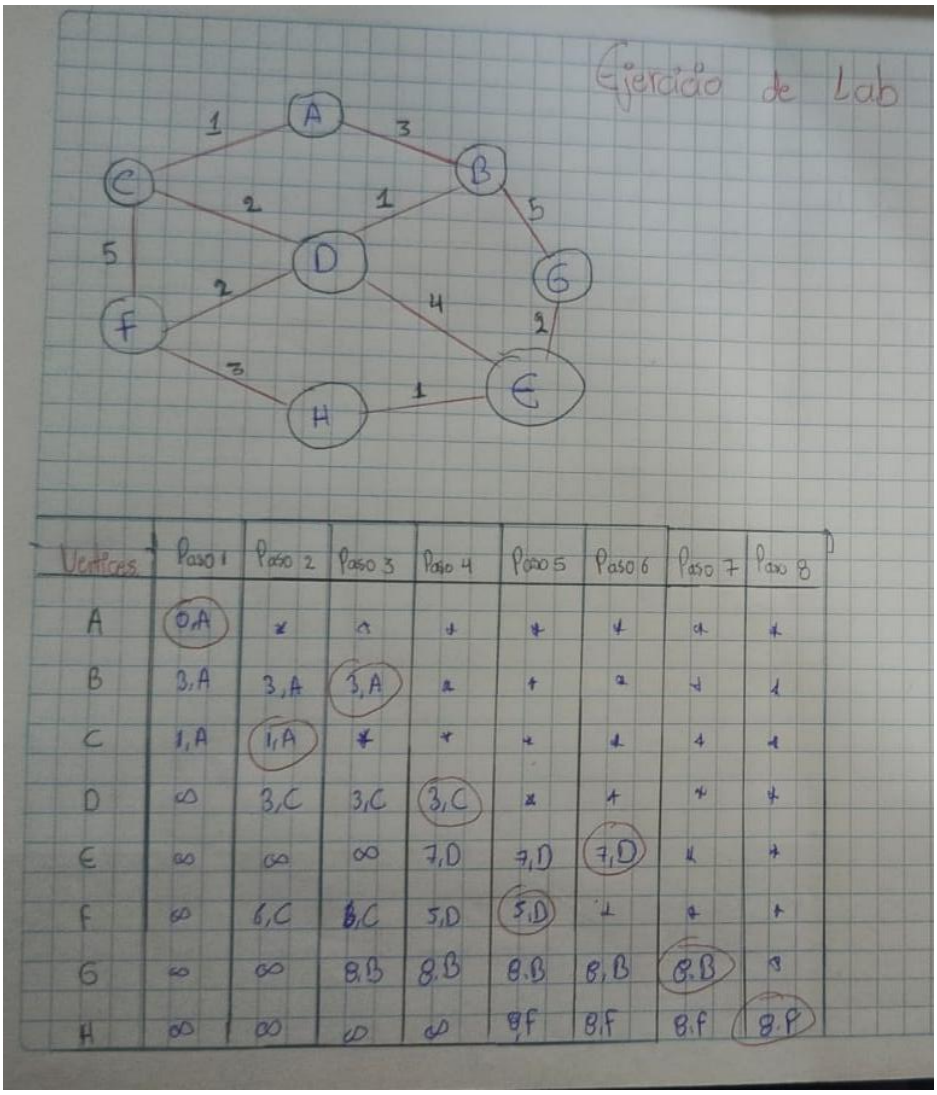
Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4.ª ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. Numerische Mathematik, 1(1), 269-271. https://doi.org/10.1007/BF01386390

Levitin, A. (2012). Introduction to the design and analysis of algorithms (3.ª ed.). Pearson.

Rosen, K. H. (2019). Discrete mathematics and its applications (8.ª ed.). McGraw-Hill.

Anexo



Vertices	Paso 1	Paso 2	Paso 3	Paso 4	Paso 5	Paso 6	Paso 7	Paso 8
A	0,A	x	x	x	x	x	x	x
B	3,A	3,A	3,A	x	x	x	x	x
C	1,A	1,A	x	x	x	x	x	x
D	∞	3,C	3,C	3,C	x	x	x	x
E	∞	∞	∞	7,D	7,D	7,D	x	x
F	∞	6,C	6,C	5,D	5,D	x	x	x
G	∞	∞	8,B	8,B	8,B	8,B	8,B	x
H	∞	∞	∞	∞	8,F	8,F	8,F	8,F